

Comparing Machine Learning Classifiers for Breast Cancer Diagnosis

Chuanchuan He

Dataset URL:

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data?resource=download>

Introduction and Motivation:

Breast cancer is one of the most important medical diagnosis problems because early and accurate detection can strongly affect patient outcomes. It is an important real-world binary classification problem where machine learning models can assist in distinguishing malignant tumors from benign tumors. In a diagnostic setting, a model that misses malignant cases can create serious clinical risk. Therefore, this project will not only focus on overall classification performance of different models but also on recall, also called sensitivity, because recall measures how well the actual malignant cases are correctly identified.

Dataset:

The dataset used in this project is the **Wisconsin Breast Cancer Diagnostic Dataset**. It contains measurements computed from the fine needle aspirate samples of breast masses. These features describe characteristics of cell nuclei.

The dataset contains:

- **569 samples (357 benign, 212 malignant)**
- **30 numerical features**
- **2 classes**
 - **Malignant**
 - **Benign**

Because the dataset is slightly imbalanced, accuracy alone may be misleading. Therefore, recall and F1-score provide more informative measures of model performance, particularly for evaluating the model's ability to correctly identify malignant cases. This is especially important in breast cancer diagnosis, where identifying false negatives (missed malignant cases) is more critical.

Approaches:

In this project, I compared three machine learning classifiers on the Wisconsin Breast Cancer Diagnostic dataset: **K-Nearest Neighbors (KNN)**, **Logistic Regression**, and a **Neural Network**. These models were selected to represent increasing levels of complexity, from a distance-based method (KNN), to a linear model (Logistic Regression), and a non-linear model (Neural Network). KNN and Logistic Regression are implemented from scratch, and the Neural Network is implemented using PyTorch. All models were evaluated using **stratified 5-fold cross-validation**, so that each fold preserves the original class distribution of malignant and benign samples. This ensures a fair and reliable comparison across models and reduces the

effect of randomness from a single train-test split. For example, Fig. 1 shows that the neural network exhibits consistent convergence behavior across all folds. Performance was measured using **accuracy, precision, recall, and F1-score**, and the final results were reported as the **mean and standard deviation across the five folds**.

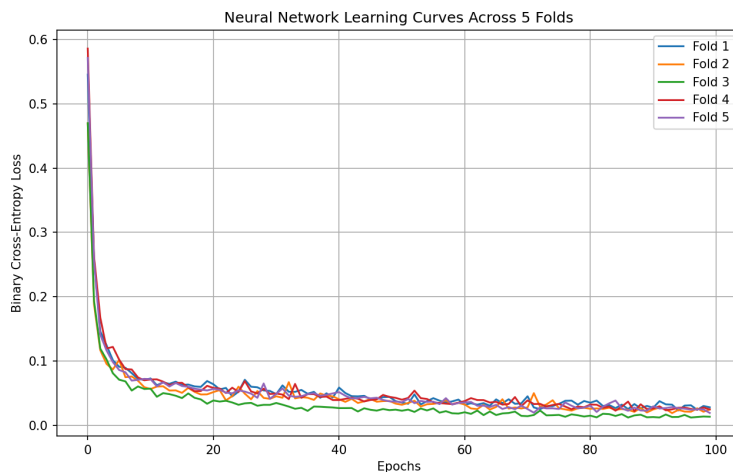


Fig.1: Neural Network learning curves across 5 folds

Computational Challenges:

One computational challenge is that the dataset contains numerical features with different scales. Since KNN is distance-based and both logistic regression and neural networks use gradient-based learning, feature scaling is important. Without scaling, features with larger numerical ranges could dominate the Euclidean distance in KNN and could also slow or destabilize gradient descent in logistic regression and neural network training. To address this issue, I applied **z-score normalization (standardization)**, where each feature is transformed using the mean and standard deviation. Importantly, **to avoid data leakage, the mean and standard deviation were computed using only the training data within each fold of cross-validation, and then applied to the corresponding test data**. Another computational challenge is selecting appropriate hyperparameters such as learning rate and batch size. Improper choices can lead to slow convergence or unstable training. This was addressed by performing hyperparameter tuning using learning curves, where the training loss was monitored over iterations/epochs to evaluate convergence behavior across different settings.

Model Architectures:

KNN: a non-parametric, instance-based model that does not explicitly learn model parameters during training. Instead, it stores the training data and makes predictions based on similarity.

- Distance metric: **Euclidean distance**
- Number of neighbors: **k = 5**
- Prediction method: majority voting among k nearest neighbors

Logistic regression: as a linear probabilistic model for binary classification, it computes a linear combination of input features and transforms it into a probability using the sigmoid function.

- Model formulation: $z = Xw + b$; $p = \frac{1}{1 + e^{-z}}$
- Loss function: **Binary Cross-Entropy Loss**
- Optimizer: **Gradient Descent**
- Learning rate (lr): **0.1**

The model outputs the probability that a sample belongs to the malignant class, and a **threshold (default = 0.5)** is applied to obtain class labels. One key hyperparameter for Logistic Regression is the **learning rate**, which controls the step size during gradient descent. This parameter was tuned using **learning curves**, where the training loss was monitored over iterations to evaluate convergence speed and stability. Based on the analysis, an appropriate learning rate of 0.1 was selected for the final experiments.

Neural Network: The Neural Network is a feedforward fully connected network implemented using PyTorch. It introduces non-linearity and allows modeling more complex relationships in the data.

- Input layer: **30 neurons**
- Hidden layer: **16 neurons**
- Activation function: **ReLU**
- Regularization: **Dropout (p = 0.2)**
- Output layer: **1 neuron**
- Output activation: **Sigmoid**
- Loss function: **Binary Cross-Entropy Loss**
- Optimizer: **Stochastic Gradient Descent (SGD) with momentum (0.9) and lr = 0.01**
- Training strategy: **Mini-batch training with Batch Size of 16**

The Neural Network includes multiple hyperparameters, including learning rate and batch size. These parameters were tuned using learning curves, where the training loss was monitored across epochs to assess convergence behavior. The final configuration (**lr = 0.01, batch size = 16**) was selected based on achieving stable and efficient convergence.

Hyperparameter Tuning:

The logistic regression learning rate tuning experiment (Fig.2) showed that the learning rate strongly affects convergence. A very small learning rate such as 0.001 resulted in slow convergence. A larger learning rate such as 0.1 converged faster and achieved a lower training loss. Therefore, **learning rate 0.1 was selected for final logistic regression evaluation.**

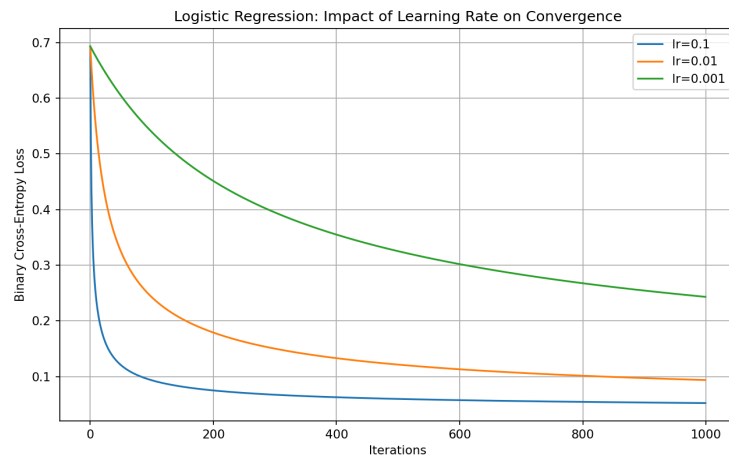


Fig.2: Logistic Regression learning rate (lr) tuning

For the neural network, the learning rate tuning experiment (Fig. 3) also showed that the learning rate affects convergence speed and stability. Among the tested values (0.001, 0.01, and 0.1), the learning rate of 0.01 achieved a lower training loss and exhibited a smoother convergence curve over 100 epochs. In contrast, the other learning rates showed more fluctuation in the loss curves, indicating less stable training. Therefore, **learning rate 0.01 was selected for final Neural Network evaluation.**

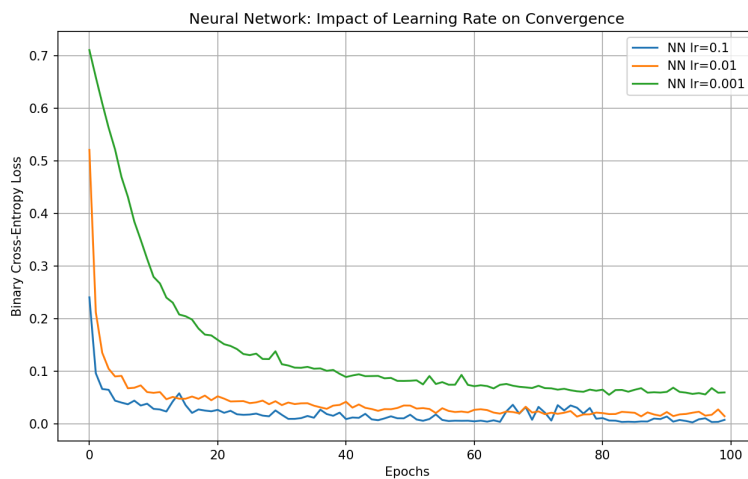


Fig.3: Neural Network learning rate (lr) tuning

The Neural Network batch size tuning experiment (Fig. 4) compared batch sizes of 16, 32, and 64 over 100 epochs. Among the tested values, a batch size of 16 achieved the lowest training loss and exhibited the smoothest convergence curve. In contrast, larger batch sizes showed slightly higher loss and less stable convergence. Therefore, **a batch size of 16 was selected for the final model.**

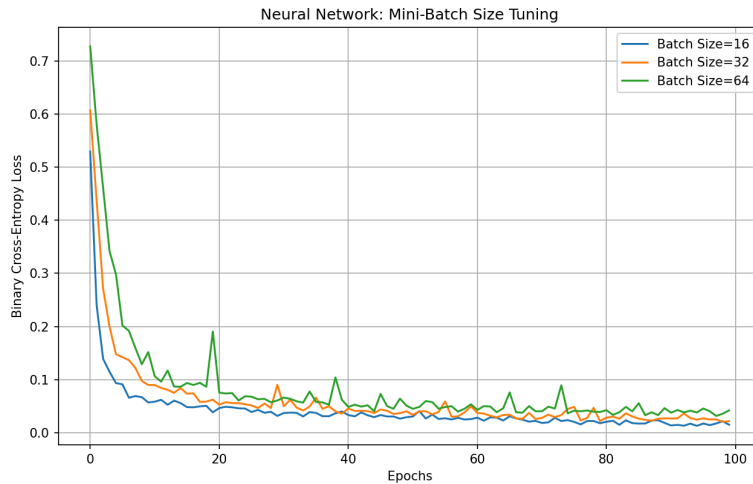


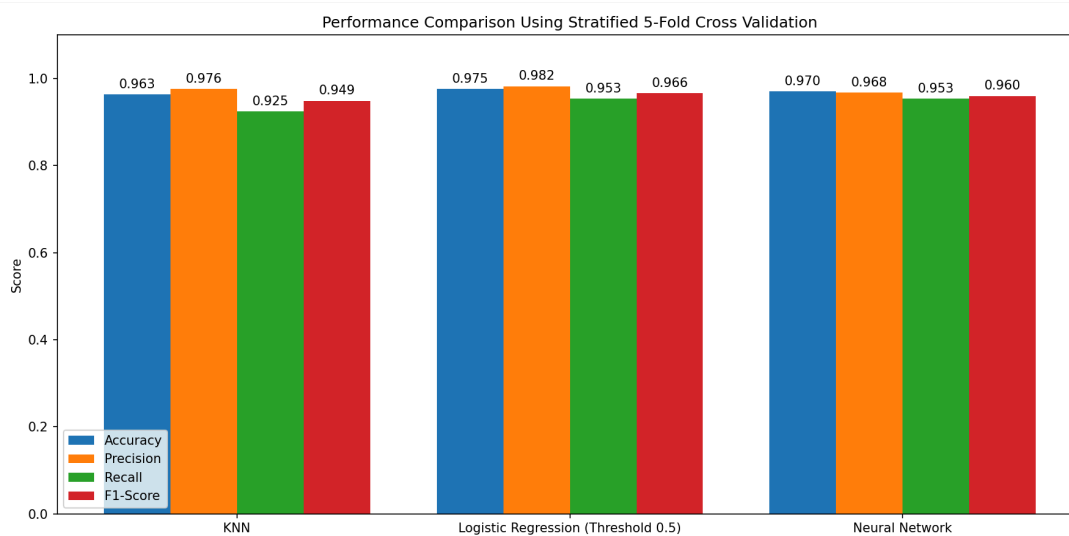
Fig.4: Neural Network batch size tuning

Results and Discussion:

The performance of the three models was evaluated using stratified 5-fold cross-validation. The results are summarized in the following table, reported as mean $\pm$ standard deviation across the five folds.

Model	Accuracy	Precision	Recall	F1-Score
KNN	0.9631 $\pm$ 0.0179	0.9755 $\pm$ 0.0217	0.9246 $\pm$ 0.0458	0.9487 $\pm$ 0.0255
Logistic Regression	0.9754 $\pm$ 0.0140	0.9820 $\pm$ 0.0260	0.9530 $\pm$ 0.0417	0.9664 $\pm$ 0.0194
Neural Network	0.9702 $\pm$ 0.0180	0.9676 $\pm$ 0.0274	0.9530 $\pm$ 0.0417	0.9596 $\pm$ 0.0248

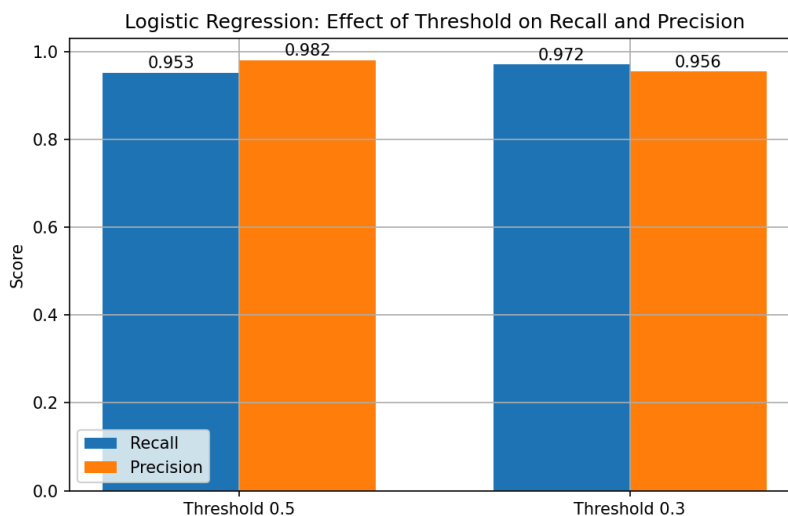
The same results are visualized in the bar plot below for easier comparison across models and evaluation metrics.



According to the results, Logistic Regression achieved the best overall performance, with the highest accuracy (0.9754 ± 0.0140), precision (0.9820 ± 0.0260), and F1-score (0.9664 ± 0.0194). It also achieved strong recall (0.9530 ± 0.0417), indicating that it was effective at correctly identifying malignant cases (reducing false negatives). This suggests that the dataset is likely close to linearly separable, allowing a simple linear model to perform very well. The Neural Network performed competitively, with accuracy (0.9702 ± 0.0180) and F1-score (0.9596 ± 0.0248) slightly lower than Logistic Regression, but still strong overall. Notably, it achieved the same recall (0.9530 ± 0.0417) as Logistic Regression, indicating that both models are similarly effective in detecting malignant cases (reducing false negatives). However, the Neural Network did not outperform Logistic Regression, likely due to the relatively small dataset size, where simpler models often generalize well without requiring complex nonlinear representations. Overall, Logistic Regression provides the best balance between performance, simplicity, and interpretability. While the Neural Network offers comparable performance, its increased complexity does not provide a clear advantage on this dataset. These results highlight that simpler models can be highly effective for structured biomedical data, and model selection should consider both performance and interpretability.

Threshold Analysis in Logistic Regression:

One way to further increase recall in Logistic Regression is to lower the classification threshold, allowing the model to classify more samples as malignant. This helps reduce false negatives, meaning fewer malignant cases are missed, which is especially important in medical diagnosis. In the experiment, lowering the classification threshold from 0.5 to 0.3 increased recall, allowing the model to detect more malignant cases, as shown in the bar plot below. However, this resulted in a decrease in precision due to an increase in false positives. This trade-off is acceptable in medical diagnosis, where minimizing false negatives is more critical than avoiding false positives. This demonstrates that the optimal classification threshold should be selected based on the application context rather than relying solely on the default threshold of 0.5.



Conclusion and Future Work:

In this project, I compared KNN, Logistic Regression, and a Neural Network for breast cancer diagnosis using the Wisconsin Breast Cancer Diagnostic dataset. The models were evaluated using stratified 5-fold cross-validation with accuracy, precision, recall, and F1-score. Among the three models, Logistic Regression achieved the best overall performance while maintaining strong recall, indicating effective detection of malignant cases. Although the Neural Network performed competitively, its increased complexity did not provide a significant advantage on this relatively small structured dataset. The threshold analysis further demonstrated that lowering the classification threshold can improve recall and reduce false negatives, which is especially important in medical diagnosis. Overall, the results show that simpler and more interpretable models can perform very effectively on structured biomedical datasets, and that model selection should consider not only predictive performance but also interpretability and clinical relevance. For future work, more advanced deep learning approaches such as **Convolutional Neural Networks (CNNs)** could be explored using medical **imaging datasets such as mammograms or histopathology images**. Unlike the current structured tabular dataset, CNNs are specifically designed for image analysis and may capture spatial patterns associated with malignant tumors more effectively. Additional future directions include evaluating more machine learning models, incorporating ROC/AUC analysis, and testing the models on larger external datasets.